11. Processes: ps, top, /proc, signals

11. Processes: ps, top, /proc, signals

Phase: Phase 4 — Processes and jobs
Estimated time: 1–2 days
Slug: `11-processes`

Learning objectives

- List and interpret processes
- Read process state from /proc
- Send signals with kill
- Understand PID, PPID, zombie/orphan concepts

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

Every running program is a *process*. When a service hangs, a script eats the CPU or a disk fills up, you need to list processes, understand their state, and stop them correctly. Choosing the right signal decides whether a program shuts down cleanly or loses data.

Core concepts

A process has a PID, a parent PID (PPID), an owner, a state, and resource usage. New processes are made by `fork` and then `exec` a program; PID 1 (`init`/`systemd`) is the ancestor of all others. When a parent dies, children get re-parented to PID 1.

State	Letter in `ps`	Meaning
Running / runnable	`R`	On CPU or in run queue
Sleeping (interruptible)	`S`	Waiting for an event
Uninterruptible sleep	`D`	Waiting on I/O, cannot be killed
Stopped	`T`	Paused by SIGSTOP/Ctrl-Z
Zombie	`Z`	Finished, parent has not collected exit status

Signals are small messages sent to processes. Some can be caught or ignored, but SIGKILL and SIGSTOP cannot.

Signal	Number	Default / use
SIGHUP	1	Terminal closed; often “reload config”
SIGINT	2	Ctrl-C interrupt
SIGQUIT	3	Ctrl- quit with core dump
SIGKILL	9	Immediate kill, cannot be caught
SIGTERM	15	Polite “please terminate” (default of `kill`)
SIGSTOP	19 (x86)	Pause, cannot be caught
SIGCONT	18 (x86)	Resume

```
 kill PID            kill -9 PID
    |                    |
 SIGTERM(15)         SIGKILL(9)
    |                    |
process handler:     kernel removes process
flush, cleanup,      immediately; no cleanup,
exit(0)              temp files and locks remain
```

Always try TERM first, wait a moment, then KILL as last resort. Tools: `ps` (snapshot), `top`/`htop` (live), `pgrep`/`pkill` (by name), `kill`, `killall`, and `/proc/PID/` for details.

Worked examples

Snapshots:

```
ps aux | head -n 3
# sample output: USER  PID %CPU %MEM    VSZ   RSS TTY  STAT START TIME COMMAND
# sample output: root    1  0.0  0.1 168000 11000 ?    Ss   09:00 0:02 /sbin/init
ps -eo pid,ppid,stat,cmd --sort=-%cpu | head -n 3
# sample output:   PID  PPID STAT CMD
```

Find and signal a process:

```
sleep 300 &
# sample output: [1] 4321
pgrep -a sleep
# sample output: 4321 sleep 300
kill 4321
# sample output: [1]+  Terminated   sleep 300
```

Signal list and numbers:

```
kill -l | head -n 3
# sample output:  1) SIGHUP   2) SIGINT   3) SIGQUIT   4) SIGILL   5) SIGTRAP
kill -l TERM
# sample output: 15
kill -l KILL
# sample output: 9
```

Exit codes after signals:

```
sleep 100 & pid=$!; kill -9 $pid; wait $pid; echo $?
# sample output: 137
```

137 is 128 + 9.

Handling a signal in a script:

```
trap 'echo cleaning up; exit 0' TERM
# sample output: (script prints "cleaning up" when it receives SIGTERM)
```

Inspect details:

```
cat /proc/$$/status | head -n 3
# sample output: Name:  bash
# sample output: Umask: 0022
# sample output: State: S (sleeping)
```

Mini-lab

1. Start `sleep 600 &` and note the PID; run `ps -o pid,ppid,stat,cmd -p PID`.
2. Run `pgrep -a sleep` and `pkill sleep`; confirm with `pgrep sleep` (no output, exit status 1).
3. Start `sleep 600 &` again; send `kill -STOP PID`, check `STAT` shows `T`, then `kill -CONT PID`.
4. Send `kill -TERM PID`; then start another and use `kill -9 PID`; compare `echo $?` after `wait`.
5. Run `ps -eo pid,ppid,cmd --forest | head -n 20` to see the tree.
6. Write a script with `trap 'echo got TERM; exit' TERM; while :; do sleep 1; done`; run in background and `kill` it.
7. Try `trap '' KILL` and note that KILL cannot be trapped.
8. Open `top`, press `P` to sort by CPU, `M` for memory, `q` to quit.

Decision guide / failure modes

Use `kill PID` (TERM) first so programs can clean up; use HUP to reload daemons that support it; use `kill -9` only for hung processes after TERM fails. Prefer `pkill -f` carefully, with `pgrep -af` to preview matches. Use `systemctl stop` for services rather than killing them by hand.

Common failures:

- `kill -9` leaves lock files, temp data and child processes.
- `pkill python` killing unrelated processes; preview with `pgrep -a`.
- Processes in `D` state ignore every signal, even KILL; the fix is the I/O cause.
- Zombies cannot be killed; fix or restart the parent.
- Confusing PID with job number (`%1`); `kill %1` targets a job, `kill 1` targets init.
- Operation not permitted: you do not own the process; use `sudo` carefully.

Common mistakes

- kill –9 as first resort
- Misreading CPU% in top
- Ignoring zombie processes as always urgent (often waitpid missing)

Exercises

Work in `exercises/11-processes/`. Read that folder's `README.md` fully before coding.

```
cd exercises/11-processes
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Implement signal_num(name). The input is a signal name as a string, with or without the "SIG" prefix and in any letter case, for example "TERM", "SIGKILL" or "int". Return the standard Linux signal number as an int: HUP=1, INT=2, QUIT=3, KILL=9, TERM=15. Specifically TERM must return 15 and KILL must return 9. For any other name return –1. Strip surrounding whitespace first. These numbers are the same ones printed by kill -l NAME, and they help you read exit statuses such as 137 (128 + 9) and 143 (128 + 15).

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/11-processes/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. TERM vs KILL?
2. What is PPID?
3. What does /proc contain?

Further reading

- https://man7.org/linux/man-pages/man1/ps.1.html